

James Barnhill

Student ID: 011148322

D214: Capstone

October 27, 2024

Capstone Report

Research question

Quarterbacks are generally viewed as the most important player on a football team. As such, high performing young quarterbacks usually sign a massive contract extension when their rookie deal expires. These contracts can be worth over 20% of the team's salary cap. For context, an NFL team's active roster consists of 53 players. In recent years, there have been a number of quarterbacks suffering season ending injuries shortly after signing these contract extensions. For example, Joe Burrow signed a contract¹ that paid him \$55M per year in 2023 (24.5% of the Cincinnati Bengals' salary cap), and tore his wrist ligament in game 10², leaving the Bengals without their starting quarterback for 7 games. Kyler Murray signed a \$46.1M per year contract in 2022 (22.1% of the Arizona Cardinals' salary cap), and tore his ACL during game 13. He didn't return until game 10 of the 2023 season, meaning the Cardinals were without their starting quarterback for 13 games. These are games in which the highest paid player on the team is unable to affect the outcome of the game. From a business perspective, these huge contracts become a huge waste of money when the players they are paying are unable to compete for significant time stretches.

Generally, the team's offensive line is seen as responsible for protecting the quarterback. A stronger offensive line will allow less sacks to the quarterback, and give the quarterback more time to throw the ball, leading to less quarterback scrambles, both of which reduce the risk of injury to the quarterback. This project seeks to determine if the modern quarterback position is being overvalued by NFL teams, at the expense of the offensive line (though I will consider the other positions as well during the analysis), and if this overvaluation leads to a higher risk of quarterback injury.

The value of an NFL player is intrinsically linked to the performance of the team as a whole. Earning a higher salary in the NFL comes with the expectation that the player contributes talents on the field that increase the team's chances of winning games as compared to the case in which the player does not participate. This expectation leads us to the overarching

research question that guides this project: ***can an analysis of NFL player salaries be used to predict team performance (win-loss records)?***

This analysis will consist of three parts as I explore this question. First, I will investigate the relationship between quarterback and offensive line salaries. This will determine if there is a negative correlation between the two positions. If there is such a negative correlation, this will indicate that teams that spend large percentages of their salary cap on quarterbacks do so by directly sacrificing offensive line value. Second, I will investigate the relationship between offensive line salaries and starting quarterback injuries. If there is a correlation *and* there is a correlation between quarterback and offensive line salaries, this could indicate that quarterbacks who get paid higher salaries are more at risk for injury due to the team's sacrifice of a highly paid offensive line. Finally, I will create a predictive model that analyses all of the team's salary data, and attempts to predict the win-loss record for the team.

The hypothesis I will be investigating is: ***There is a statistically significant correlation between NFL position salaries and win-loss records.*** The null hypothesis is: ***There is not a statistically significant correlation between NFL position salaries and win-loss records.*** This hypothesis is useful to investigate for NFL teams, as rejecting the null hypothesis would indicate to teams that strategizing how different positions are paid as proportions of the team's salary cap can lead to better end of season performance.

Data Collection

For this project, I collected data from each NFL team from the last 10 complete seasons: 2014-2023. Styles of play evolve quickly in professional sports, and it would not make sense to consider data from previous eras of the NFL, in which different positions had stronger or weaker roles in the game than they do today.

The first data set I collected gave end of season records for each team by year. This data was collected from pro-football-reference.com. The below screenshot shows a selection of this data for the AFC East teams in 2023.³

Tm	W	L	W-L%	PF	PA	PD	MoV	SoS	SRS	OSRS	DSRS
AFC East											
Buffalo Bills *	11	6	.647	451	311	140	8.2	-1.8	6.5	4.1	2.3
Miami Dolphins +	11	6	.647	496	391	105	6.2	-1.8	4.4	7.6	-3.1
New York Jets	7	10	.412	268	355	-87	-5.1	-0.6	-5.8	-6.7	0.9
New England Patriots	4	13	.235	236	366	-130	-7.6	-0.2	-7.9	-8.5	0.6

From this data set I used the team column ('Tm') to identify the team, the win-loss percentage column ('W-L%') to evaluate the end of season performance, and the strength of schedule column ('SoS') to evaluate the performance of the team's opponents. Note that I specifically used win-loss percentage, rather than number of wins. This is for two reasons: first, on rare occasion there are ties in the NFL, which get weighted as half of a win for win-loss percentage, and would be weighted as a loss if I only considered number of wins. Second, the NFL expanded the season from 16 games to 17 games in 2021⁴ so using the win-loss percentage allows the comparison of season records before and after this change.

I considered the strength of schedule variable as a way to account for the fact that wins and losses of an NFL team are dependent on the strength of its opponents. A mediocre team will have an excellent season record if all of its opponents are bad, and an excellent team could have a mediocre season record if all of its opponents are also excellent. The rest of the variables taken from this data set are not used for the analysis however, as they are all related to the strength of the team (e.g. a team with a high points for ('PF') value and a low points allowed ('PA') value, like the Buffalo Bills in the above image, will very likely have a good end of season record, because they scored more than they allowed their opponents to score). This would confound the goal of attempting to predict the end of season record based on position spending.

The second data set I collected contains salary data for each player on an NFL team by year. This data was collected from [spotrac.com](https://www.sportstrac.com). The below screenshot shows a portion of the data collected for the 2023 Miami Dolphins.⁵

2023 Active Roster Cap

■ VOID YEAR ■ JUNE 1ST CUT

PLAYER (53)	POS	AGE	CAP HIT	CAP HIT PCT LEAGUE CAP	DEAD CAP	BASE PS SALARY
Emmanuel Ogbah	DE	30	\$17,185,292	7.64%	(\$21,000,000)	\$15,000,000
Tyreek Hill	WR	29	\$12,823,750	5.70%	(\$66,165,000)	\$1,165,000
Jerome Baker	ILB	27	\$12,516,111	5.57%	(\$17,149,444)	\$8,410,000
Christian Wilkins	DT	28	\$10,753,000	4.78%	(\$10,753,000)	\$10,753,000
Xavien Howard	CB	30	\$10,071,281	4.48%	(\$37,176,562)	\$1,165,000

From this data I collected the player name, their position ('POS'), and their cap hit. NFL teams are limited in their spending by a salary cap. Rather than analyze the intricacies of each

player's contract, which contain various incentives and bonuses, I simply took the overall hit to the team's cap, as this represents how the team budgets their spending. Additionally, I took the data from the total salary table, not just the active roster, as team spending on injured reserve players and practice squad players is also important to consider for the analysis of overall teams spending.

One challenge I encountered while collecting this data was that the tables for the 2021 Miami Dolphins and the 2021 Los Angeles Chargers would not load on this website. The web scraping tool I used (discussed further below) could not obtain the data for these teams, and going to the website itself either timed out or got stuck on a loading screen, shown below.⁶

The screenshot shows a web interface for team selection. At the top, there are two dropdown menus: the first is set to '2021' and the second is set to 'LAC Chargers'. Below these are three summary boxes: 'TOTAL CAP ALLOCATIONS' showing '\$187,797,479 / 16th', 'CAP SPACE' showing '\$579,480 / 27th', and 'RESERVE LISTS' showing '\$10,985,000 / 28th'. Below the summary boxes is a section titled '2021 Active Roster Cap'. Under this title, there are two legend items: 'VOID YEAR' with an orange square and 'JUNE 1ST CUT' with a red square. Below the legend is a table that is partially obscured by a 'Filtering & Sorting...' overlay with a circular refresh icon.

Ultimately, I decided that this represented two data points out of the 320 points I attempted to collect. As this was the only missing data encountered for this project, I determined that it was acceptable to remove these two data points from the analysis, as it only represented 0.6% of the total data.

The third and final data set I collected was a timeline of starting quarterback changes for each team. This data was collected from quirkyresearch.com. An example of the start of the Baltimore Ravens' timeline is shown in the screenshot below.⁷

Cincinnati Bengals (1968-2024)

Name ^{stint}	Tenure [Year – Week (Game)]		GS	Reason for Change From	* Record
Starting QB	First Start	Last Start			
Dewey Warren	1968 – 1 (1)	1968 – 1 (1)	1	ineffective	0-1
John Stofa	1968 – 2 (2)	1968 – 7 (7)	6	ineffective	2-4
<i>Warren</i> ²	1968 – 8 (8)	1968 – 8 (8)	1	concussion	0-1
Sam Wyche	1968 – 9 (9)	1968 – 11 (11)	3		* 1-2
<i>Warren</i> ³	1968 – 12 (12)	1968 – 12 (12)	1	ineffective	0-1
<i>Stofa</i> ²	1968 – 13 (13)	1968 – 13 (13)	1	ineffective	0-1
<i>Warren</i> ⁴	1968 – 14 (14)	1968 – 14 (14)	1	Cook drafted 1969 #5 overall; Stofa and Warren released in training camp	0-1
Greg Cook	1969 – 1 (1)	1969 – 3 (3)	3	torn right rotator cuff	3-0
<i>Wyche</i> ²	1969 – 4 (4)	1969 – 4 (4)	1		* 0-1
<i>Cook</i> ²	1969 – 5 (5)	1969 – 5 (5)	1	torn right rotator cuff	0-1

In the above table, note the asterisk to the left of the record column for the two starting tenures of Sam Wyche. This asterisk indicates that the ‘usual starter’ returned after this quarterback’s tenure was finished. For this data, I only consider injuries to the ‘usual starter’, as they are the player that would presumably get paid the most. I use this asterisk as a way to track which quarterback is considered the ‘usual starter’, and then find the reason they left the game. For example, looking at Wyche’s second tenure in 1969 shows that Greg Cook returned as quarterback after Wyche, so looking before Wyche’s tenure to Cook’s previous tenure shows that Cook left due to a torn right rotator cuff. One challenge I encountered here is that it is not easy to build code that would determine if the reason a quarterback left was due to an injury or not. Instead, I made a copy of the csv file I generated from the data, opened it in Excel, and hand labeled each row in the data as to whether or not the reason was an injury.

To scrape all the data used for this project, I used the html scraping tool BeautifulSoup from the python library bs4. The code I wrote using this scraper was inspired by code found on a github repository used for scraping spotrac.com.⁸ The advantage of this tool is that it quickly scrapes all the html data from a website. Since each site is formatted the same for different teams/years, this means that it is easy to loop code over the total domain of data (every team for each year in 2014-2023) and pull the same data from each site. Ultimately, my code only takes between 10-15 minutes to collect all of the data for this project. The disadvantage of this

tool is that websites have a lot of html data that is not useful. It takes time to sort through all of the html data to find the relevant data to save, and then write code that will save only the relevant data.

Data Extraction and Preparation

For the first data set I extracted the useful data using a BeautifulSoup object as an html parser. I isolated each data table on the website by using the find_all method with the argument 'tbody'. For each table, I found each row of the table using the find_all method with the argument 'tr'. Then I found the value of each cell in the row by using the find_all method with the arguments 'td' and 'th'. The text value of each element was then saved to a list. As some years did not have a tie column, (due to no ties during the season), I had to create a tie column with values 0 for each team so that each year would have the same number of columns for the purpose of concatenating them into one large data frame.

```
url = "https://www.pro-football-reference.com/years/{}/".format(year)
season_records = requests.get(url).text
season_soup = BeautifulSoup(season_records, "html.parser")

conference_tables = season_soup.find_all('tbody')

all_rows = []
column_names = ['Team', 'Win', 'Loss', 'Tie', 'Win_Loss', 'Points_For', 'Points_Allowed',
                'Point_Differential', 'Margin_of_Victory', 'Strength_of_Schedule', 'SRS',
                'Off_SRS', 'Def_SRS']
for c in conference_tables:
    rows = c.find_all('tr')
    for r in rows:
        elements = r.find_all(['td', 'th'])
        element_list = []
        for e in elements:
            element_list.append(e.text)
        if len(element_list) > 1:
            if len(element_list) < len(column_names):
                #Some years don't have ties, and the column is left off.
                #Add in a tie column if not present
                element_list.insert(3, '0')
            all_rows.append(element_list)
```

Next, I performed some initial cleaning on the data. I striped any special characters from the team names (which were used to indicate playoff positions), I added a year column (since I later combine all the data into one single data frame), and set the appropriate data types for each column, since they are scraped as strings.

```

year_results_df = pd.DataFrame(all_rows, columns = column_names)
#remove special characters from team names which indicate playoff positions
year_results_df['Team'] = [team if team[-1].isalpha() else team[:-1] for team in year_results_df['Team']]
year_results_df['Year'] = [year for i in year_results_df['Team']]

#set appropriate data types for each column
ints = ['Win', 'Loss', 'Tie', 'Points_For', 'Points_Allowed', 'Point_Differential']
floats = ['Win_Loss', 'Margin_of_Victory', 'Strength_of_Schedule', 'SRS', 'Off_SRS', 'Def_SRS']
year_results_df[ints] = year_results_df[ints].astype('int')
year_results_df[floats] = year_results_df[floats].astype('float')

```

This code was all placed into a function called `get_yearly_results(year)`, which I then looped over for each year in question, and then concatenated them all together.

```

years = np.arange(2014, 2024)

#get all end of season results for every year

dataframes = []

for y in years:
    results = get_yearly_results(y)
    dataframes.append(results)

yearly_results = pd.concat(dataframes)
yearly_results = yearly_results.reset_index(drop = True)

```

I then performed some final cleaning on the data by changing out of date team names. The Raiders, Chargers, Rams, and Commanders have all changed team names since 2014, however the other two data sets only use the present team name for past data. For ease of analysis, I updated all team names in this data set to their present name with the following code, and then saved the data as a csv file.

```

def curr_team_name(team):
    if team == 'Oakland Raiders':
        team = 'Las Vegas Raiders'
    elif team == 'San Diego Chargers':
        team = 'Los Angeles Chargers'
    elif team == 'St. Louis Rams':
        team = 'Los Angeles Rams'
    elif team == 'Washington Redskins' or team == 'Washington Football Team':
        team = 'Washington Commanders'
    return team

yearly_results['Team'] = [curr_team_name(team) for team in yearly_results['Team']]
yearly_results
yearly_results.to_csv('yearly_results.csv', index = False)

yearly_results.head()

```

For team spending data, I used a similar strategy. I parsed the website html data using a BeautifulSoup object. I found all the tables on the page using the `find_all` method with the

argument 'table'. I looped through each table, and first analyzed the table head using find_all with the argument 'thead'. I looped through the head, and extract column names using find_all with the argument 'th'. This was done because each page has a summary table for the team, which was not needed for the analysis. The summary table does not have the column names for the data needed, so I do not save any data from a table without these column names.

```
url = "https://www.spotrtrac.com/nfl/{0}/cap/_/year/{1}/sort/cap_total".format(team, year)

response = requests.get(url).text
#pass contents of website using bs and convert to txt
soup = BeautifulSoup(response,"html.parser")

tables = soup.find_all('table')

bodies = []
for t in tables:
    head = t.find_all('thead')
    if len(head) > 0:
        cols = head[0].find_all('th')
        col_names = []
        for c in cols:
            name = re.sub('[^a-zA-Z]', ' ', c.text).strip()
            col_names.append(name)
```

Next I found all the rows in the body of the table, and looped through each row, extracting each element and saving to a list. This list is then appended to a list of lists, which is then saved as a dataframe. I then created one master data frame, and added all relevant data from each of the tables. Again, note that the summary table did not have column names, so no data saved from the summary table would be erroneously added from this table, as selecting the columns ['Player', 'Pos', 'Cap Hit'] from this table would result in an error, which would be ignored due to the exception rule, and no data would be added.

```
body = t.find_all('tbody')
rows = body[0].find_all('tr')
all_rows = []
for r in rows:
    elements = r.find_all(['td', 'th'])
    element_list = []
    for e in elements:
        element_list.append(e.text.strip())
    all_rows.append(element_list)
try:
    df = pd.DataFrame(all_rows, columns = col_names)
except:
    df = pd.DataFrame(all_rows)
bodies.append(df)

all_players = pd.DataFrame(columns = ['Player', 'Pos', 'Cap Hit'])
for df in bodies:
    try:
        data = df[['Player', 'Pos', 'Cap Hit']]
        all_players = pd.concat([all_players, data])
    except:
        None
```

Next, I performed some initial cleaning on this data. I removed the dollar sign from the cap hit column, and converted this value to an integer. On occasion, no salary data was present for a player. In these instances I assumed a salary of 0, and replaced the empty value with 0. Finally, I grouped all the data by position and took the sum. This results in table that gives the total spending at each position for the team that year. I saved this table as a dictionary, where the keys are the position, and the values are the total amount spent at that position. This is done because there are a few positions that teams don't spend on every year, such as the kick returner ('KR') position. Saving as a dictionary makes it easier to combine all the data into one large data frame in the next step.

```
all_players['Cap Hit'] = [re.sub('[^\d]', '', amount) for amount in all_players['Cap Hit']]
#if no data given for a player, assume the contribution is zero
all_players['Cap Hit'] = [0 if amount == '' else amount for amount in all_players['Cap Hit']]
all_players['Cap Hit'] = all_players['Cap Hit'].astype('int')
all_players.reset_index(inplace = True, drop = True)

pos_breakdown = all_players.groupby('Pos')['Cap Hit'].sum()
keys = pos_breakdown.index.values
values = pos_breakdown.values
pos_dict = dict(zip(keys, values))
```

The above code for this data set is all saved into a function called `get_pos_breakdown(team, year)`. I then loop over each instance saved in the first data set, pull the team name and year, convert the team name into the proper format for the [sportstrac.com](https://www.sportstrac.com) website, and scrape the data, throwing an error if the data could not be obtained. As noted before, the data for the 2021 Dolphins and Chargers could not be obtained due to the site timing out.

```
#scrape position cap data for each team per year

position_array = []
error_array = []
for i in range(len(yearly_results)):
    team = yearly_results.iloc[i]['Team']
    team = re.sub(' ', '-', team).lower()
    year = str(yearly_results.iloc[i]['Year'])

    try:
        position_results = get_pos_breakdown(team, year)
        position_array.append([team, year, position_results])

    except:
        print('Scraping error for {0} {1}.'.format(year, team))
        error_array.append([team, year])
```

```
Scraping error for 2021 miami-dolphins.
Scraping error for 2021 los-angeles-chargers.
```

Next, I looped over all the data scraped, and created a set of all possible positions that teams could spend on. I then looped through each dictionary of position spending values, and added any missing positions to that dictionary, with the value of 0. This dictionary is then converted into a data frame, which is concatenated with a master data frame containing data for each team and each year. This data frame is then saved to a csv file.

```
#create a set of all possible position labels
pos_set = set()
for i in range(len(position_array)):
    pos_dict = position_array[i][2]
    pos_keys = list(pos_dict.keys())
    pos_set.update(pos_keys)

#save all scraped position data to a dataframe, and export to a csv for later use
position_df = pd.DataFrame()
for i in range(len(position_array)):
    pos_dict = position_array[i][2]
    dict_keys = list(pos_dict.keys())
    for p in pos_set:
        if p not in dict_keys:
            pos_dict[p] = 0
    pos_dict['Team'] = position_array[i][0]
    pos_dict['Year'] = position_array[i][1]
    pos_df = pd.DataFrame([pos_dict])
    position_df = pd.concat([pos_df, position_df], ignore_index = True)

position_df = position_df.reset_index(drop = True)
position_df.to_csv('position_breakdown.csv', index = False)
```

The third data set of quarterback timelines is organized by division, not by team. So I first defined a dictionary of NFL teams by division, in the order that they appear on the quirkyresearch.com pages.

```
divisions = ['afc-east', 'afc-north', 'afc-west', 'afc-south',
            'nfc-east', 'nfc-north', 'nfc-west', 'nfc-south']

teams = {'afc-east': ['Buffalo Bills', 'Miami Dolphins', 'New England Patriots', 'New York Jets'],
        'afc-north': ['Baltimore Ravens', 'Cincinnati Bengals', 'Cleveland Browns', 'Pittsburgh Steelers'],
        'afc-west': ['Denver Broncos', 'Kansas City Chiefs', 'Las Vegas Raiders', 'Los Angeles Chargers'],
        'afc-south': ['Houston Texans', 'Indianapolis Colts', 'Jacksonville Jaguars', 'Tennessee Titans'],
        'nfc-east': ['Dallas Cowboys', 'New York Giants', 'Philadelphia Eagles', 'Washington Commanders'],
        'nfc-north': ['Chicago Bears', 'Detroit Lions', 'Green Bay Packers', 'Minnesota Vikings'],
        'nfc-west': ['Arizona Cardinals', 'Los Angeles Rams', 'San Francisco 49ers', 'Seattle Seahawks'],
        'nfc-south': ['Atlanta Falcons', 'Carolina Panthers', 'New Orleans Saints', 'Tampa Bay Buccaneers']}
```

I scraped the website data for each division using a BeautifulSoup object, and found each table on the page (one corresponding to each team in the division). Using the same methods as the above two data sets, I pulled out the elements of each row, and save to a list of lists. For this data, the first row of the table happens to be the column names, so I created a data frame using this data, with the first row being used to set the column names. I then performed some initial cleaning on the quarterback name data by removing the superscript numbers included on the website.

```
url = "https://www.quirkyresearch.com/nfl-starting-quarterback-timelines/{division}-starting-quarterback-changes/".format(division)

response = requests.get(url).text

soup = BeautifulSoup(response,"html.parser")

tables = soup.find_all('table')
div_teams = teams[division]
dfs = []
for i in range(len(tables)):
    team = div_teams[i]
    table = tables[i]

    rows = table.find_all('tr')
    start_qb_arr = []
    for r in rows:
        vals = r.find_all('td')
        vals = [v.text.strip() for v in vals]
        if len(vals) == 7:
            start_qb_arr.append(vals)

    #quarterback log dataframe
    df = pd.DataFrame(start_qb_arr[1:], columns = start_qb_arr[0])
    df['Starting QB'] = [re.sub('\d', '', qb).strip() for qb in df['Starting QB']]
```

Next, I created a log of the injuries I was interested in: injuries to the starter. I looped through the table, and identified instances in which the quarterback was followed by the return of a starter (indicated by the asterisk, as discussed in the Data Collection section of this report. I then identified the starting quarterback as the next quarterback on the list. I then looped backwards over the table, until I reached the same starting quarterback. While performing this loop, I added up the number of games that the quarterback missed while they were gone. I then saved the following data found via this analysis: the quarterback, their team, the year they left, the number of games they missed, and the reason they left. I only saved this data if they year they left was after 2013. I then concatenate all the injury logs for each team in the division.

```

injury_log = []

for j in range(len(df)):
    if df.iloc[j]['*'] == '*':
        #if this value is an asterisk, the usual starter returned the next week
        return_starter = df.iloc[j+1]['Starting QB']

        index = j
        qb = df.iloc[index]['Starting QB']
        weeks_missed = 0
        #find when the usual starter left
        while return_starter not in qb:
            weeks_missed += int(df.iloc[index]['GS'])
            index -= 1
            qb = df.iloc[index]['Starting QB']

        reason = df.iloc[index]['Reason for Change From']
        week_injured = df.iloc[index]['Last Start']
        week_returned = df.iloc[j+1]['First Start']
        year = int(week_injured[:4])
        if year > 2013:
            report = [qb, team, year, weeks_missed, reason]
            injury_log.append(report)

injury_df = pd.DataFrame(injury_log, columns = ['Quarterback', 'Team', 'Year', 'Games_Missed', 'Reason'])
dfs.append(injury_df)
return(pd.concat(dfs))

```

The above code is all saved to a function called `division_qb_changes(division)`. I looped over this function for each division, and concatenated all the data together, saving to a csv file.

```

injury_dfs = []
for d in divisions:
    injury_df = division_qb_changes(d)
    injury_dfs.append(injury_df)
all_injuries = pd.concat(injury_dfs)
all_injuries.to_csv('all_injuries.csv')

```

Next, I copied this file - `all_injuries.csv` - and hand labeled each row as to whether or not the logged quarterback change was due to an injury (1 for injured, 0 for not injured).

At this point, all the data I had collected was saved to three csv files. Next, I had to clean and combine these three csv files into one master file for analysis. First, I imported the hand labeled injury data, and filtered out all rows that weren't injuries. I then grouped the data frame by team and year, and summed over the number of games missed. This gives a value for each year as to the number of games the starting quarterback missed due to injury as a result of an injury happening that year. Note that this allows for a year to have a value larger than the number of games that season, if it took the quarterback longer than a season to return from their injury. This value would only appear for the year in which the injury happened. I then perform a left join on this table with the yearly records data set, using the team name and year as keys. When

the merge is complete, any years that did not have a starting quarterback injury will be populated with NaN values. These are replaced with 0, as there were no games missed by the starter due to injury.

```
all_injuries_labeled = pd.read_csv('all_injuries_labeled.csv')
all_injuries_filtered = all_injuries_labeled[all_injuries_labeled['Injured'] == 1][['Team', 'Year', 'Games_Missed']]
all_injuries_filtered

injuries_summed = all_injuries_filtered.groupby(['Year', 'Team']).sum('Games_Missed')
injuries_summed.reset_index(inplace = True)
injuries_summed

yr_copy = yearly_results.copy()

yearly_and_injuries = pd.merge(yr_copy, injuries_summed, on = ['Team', 'Year'], how = 'left')
yearly_and_injuries['Games_Missed'] = yearly_and_injuries['Games_Missed'].fillna(0).astype('int')
yearly_and_injuries.head()
```

Next, I prepared the position salary data to be merged with the other two sets. First, I cleaned the team names in the file so that they are in the same form as the team names for the other two sets. This is necessary, as I again used the team name and year as keys when merging. I then transformed each column of salary data to be represented as a fraction of the team's total cap spending for the year. This was done because the salary cap amount changes from year to year, so in order to compare across years this data must be used as a fraction of the total. I then merge this data with the other data, creating one master data set. Finally, I drop the Na values, which correspond to the two teams that I could not obtain salary data for (2021 Dolphins and 2021 Chargers).

```
position_breakdown = pd.read_csv('position_breakdown.csv')

def clean_team_name(name):
    cleaned = re.sub('-', ' ', name)
    cleaned = ' '.join(w.capitalize() for w in cleaned.split())
    return cleaned

position_breakdown['Team'] = [clean_team_name(team) for team in position_breakdown['Team']]

#represent each positional spending as a fraction of the team's total spending for the year
pos_columns = position_breakdown.columns[:-2]
tot_spending = position_breakdown[pos_columns].sum(axis = 1)
position_breakdown['Tot_Spending'] = tot_spending

for col in pos_columns:
    position_breakdown[col] = position_breakdown[col]/position_breakdown['Tot_Spending']

position_breakdown.drop(columns='Tot_Spending',inplace = True)

all_data = pd.merge(yearly_and_injuries, position_breakdown, on = ['Team', 'Year'], how = 'left')

#two data points could not be scraped from spotrac: 2021 Miami Dolphins and 2021 LA Chargers
#visiting the actual website shows the website timing out - there must be a site issue
#as this only accounts for 0.6% of the data, I will simply drop these two rows
all_data.dropna(inplace = True)
all_data.reset_index(drop = True, inplace = True)
```

The final step of preparation I performed was to create a variable 'O_Line', which was the sum of all the offensive line position salaries for each team. These positions are center ('C'), guard ('G'), left tackle ('LT'), right tackle ('RT'), tackle ('T'), and offensive lineman ('OL'). I then save the cleaned and prepared data to the csv file 'nfl_data.csv'.

```
#sum of all positions on the offensive line
o_line = ['C','G','LT','RT','T','OL']
all_data['O_Line'] = all_data[o_line].sum(axis = 1)

all_data.to_csv('nfl_data.csv', index = False)
all_data.head()
```

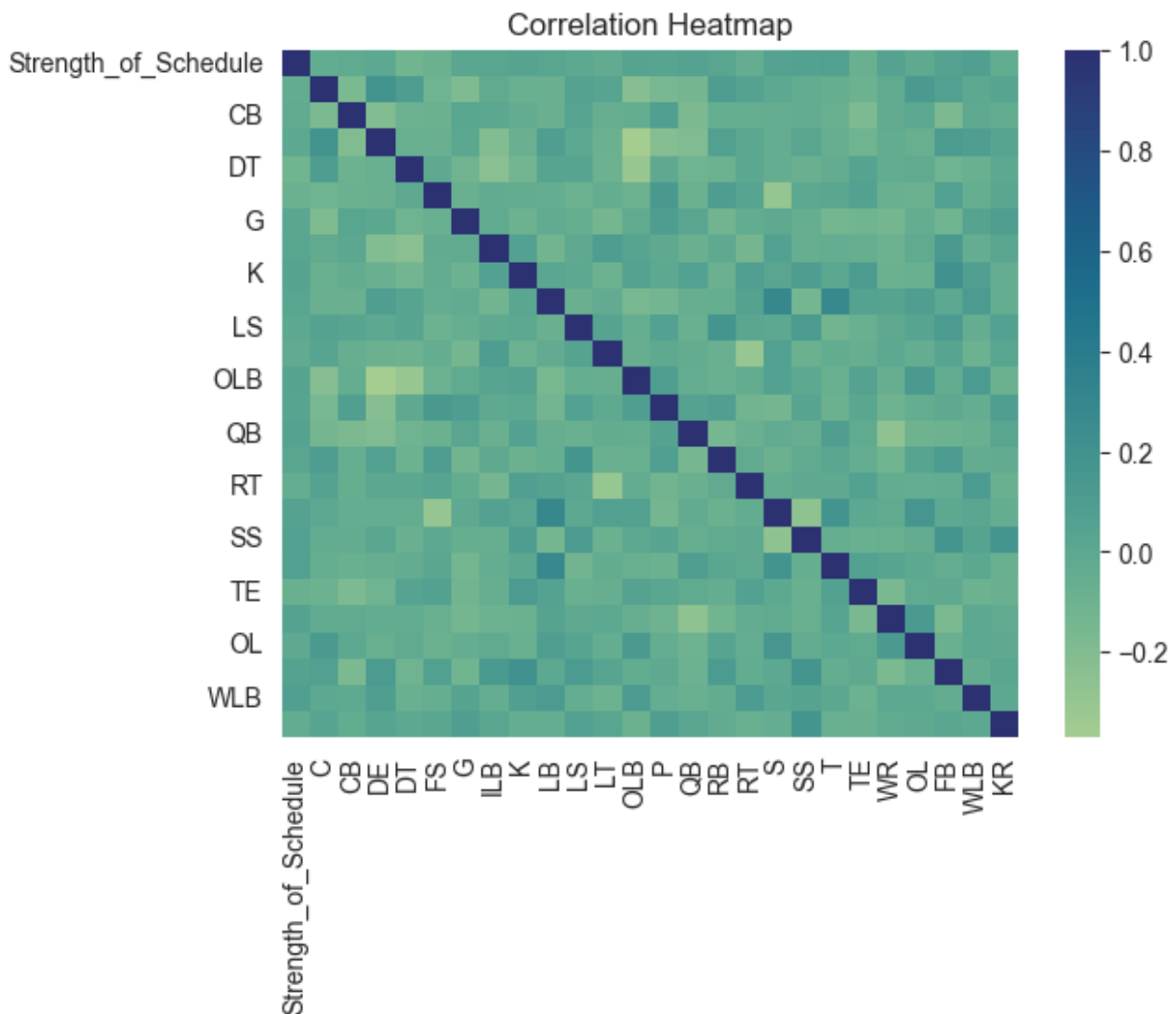
To summarize the data extraction and preparation, I used a web scraping technique to obtain html data for three web sources. Using BeautifulSoup objects, I extracted the relevant data from tables within the html data. The disadvantage of this technique was that it does not return the data in a usable state, and the actual data needs to be extracted from the html data, which is cumbersome. The advantage of this technique was that once I created a function to extract the data for one team, I was able to efficiently automate the data collection for repeated scraping of the websites, as in the case of the team salary table I had to scrape the [sportrac.com](https://www.sportrac.com) website 320 times, one for each team for each year. To clean the data I used simple python string manipulation techniques, such as regular expressions, and simple pandas data frame methods to combine all the data into one master file. One disadvantage here was that I had to hand-label the injury status of each quarterback change, as it was not labeled by the data source in a convenient manner.

Analysis

For the data analysis I selected Strength of Schedule, and all of the positional salary variables as the independent variables. I first performed a cross correlation analysis, to confirm that none of the independent variables were strongly correlated, as I want to avoid any redundancies from including two highly correlated variables when creating linear models. This analysis is displayed as a heatmap.

```
ind_vars = ['Strength_of_Schedule', 'C', 'CB', 'DE', 'DT', 'FS', 'G', 'ILB', 'K', 'LB',
            'LS', 'LT', 'OLB', 'P', 'QB', 'RB', 'RT', 'S', 'SS', 'T', 'TE', 'WR', 'OL', 'FB', 'WLB', 'KR']
corr = cleaned_data[ind_vars].corr()

#plot a heatmap of the correlation matrix
plt.figure()
plt.title('Correlation Heatmap')
sns.heatmap(corr, cmap = 'crest')
plt.show()
```



By examining this plot, the only strong correlation values are along the diagonal (variables being cross correlated with themselves), so I conclude that each variable can indeed be considered independent from the others.

Next, I separated the cleaned data into data frames containing the X data (independent variables: strength of schedule and position salary variables) and y data (dependent variable: win-loss). I added the total of the offensive line spending variable to the X data, and I added a boolean variable 'QB_Lost' to the X data, indicating if the team lost their quarterback for significant time. I used a cutoff value of 4 games to indicate significant time. I added this variable during the analysis phase rather than the data preparation phase to give the opportunity to experiment with different cutoff values, however ultimately found that changing this value did not result in any significant improvement on the regression analysis. I selected

four games for the final analysis as this is the number of games a player is required to sit out when placed on the injured reserve, and is therefore a good representation of a player missing significant time. I then create a train-test split with 80% training data and 20% testing data.

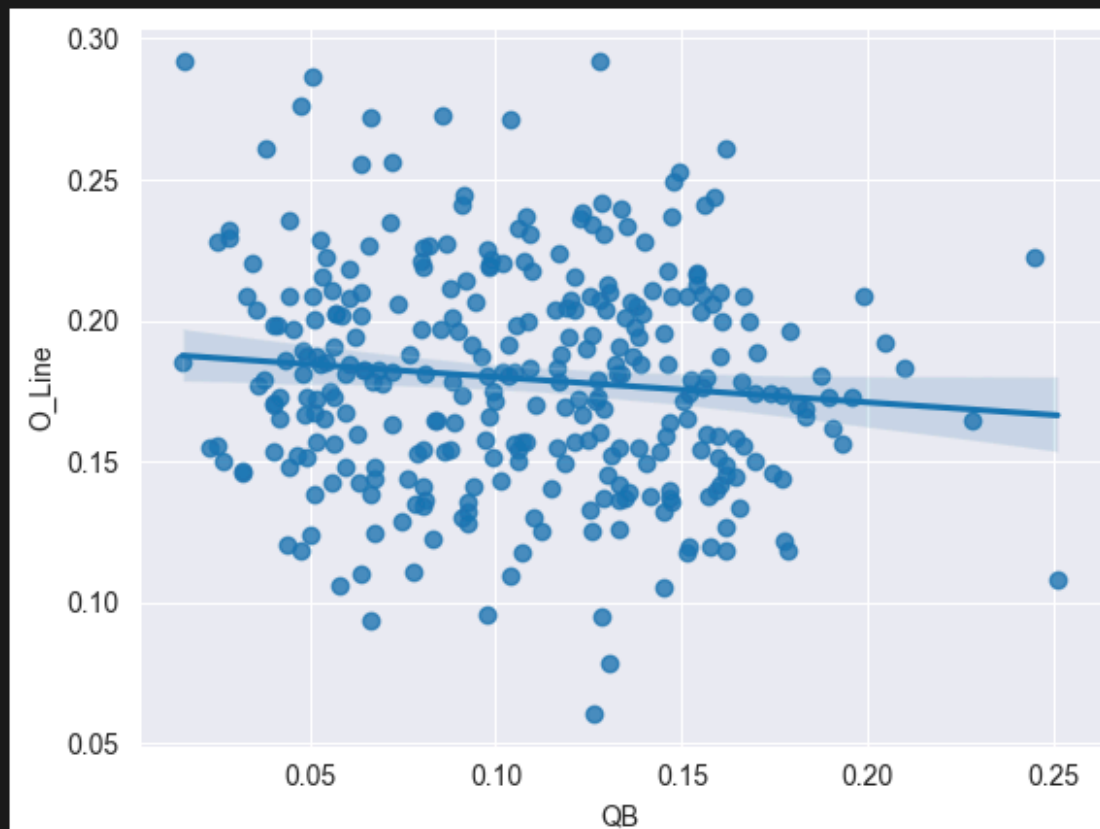
```
games_missed_cutoff = 4

x_vars = ind_vars.copy()
x_vars.append('O_Line')
X = cleaned_data[x_vars]
X['QB_Lost'] = (cleaned_data['Games_Missed'] > games_missed_cutoff).astype('int')
y = cleaned_data['Win_Loss']
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2)
train = pd.concat([X_train, y_train], axis = 1)
test = pd.concat([X_test, y_test], axis = 1)
```

Next, I performed a linear regression analysis on total offensive line spending as a function of quarterback spending. First I plotted a scatterplot of the two variables with a trend line, using the seaborn package regplot.

```
#plot a scatterplot of QB spending vs O-line spending
plt.figure()
sns.regplot(x = 'QB', y = 'O_Line', data = cleaned_data)
plt.show()
```

✓ 0.4s



Then, I trained a linear model using the statsmodels function ols on the training data. I used the testing data to predict O_line values from the model, and used the following code to calculate R-squared values and residual standard errors for the training and testing data.

```
#create a linear model of o-line spending as a function of qb spending

qb_v_oline_model = ols('O_Line ~ QB', data = train).fit()

rsquared = qb_v_oline_model.rsquared
rse = np.sqrt(qb_v_oline_model.mse_resid)
train_mean = np.mean(train['O_Line'])

#use linear model to predict O-line spending from qb spending
y_pred = qb_v_oline_model.predict(test)

#use residuals to calculate a r-squared value and rse for test group
residuals = test['O_Line'] - y_pred
test_mean = np.mean(test['O_Line'])
dist_mean = test['O_Line'] - test_mean
test_rsquared = 1 - np.sum(residuals**2)/np.sum(dist_mean**2)
test_rse = np.std(residuals)

print('R squared value of the linear model: {0:.3f}'.format(rsquared))
print('Residual standard error of the linear model: {0:.3f}'.format(rse))
print('The RSE of the linear model is {0:.3f}% of the mean.'.format(rse/train_mean*100))
print('\nR squared value of the linear model applied to the test data: {0:.3f}'.format(test_rsquared))
print('Residual standard error of the linear model applied to the test data: {0:.3f}'.format(test_rse))
print('The RSE of the linear model applied to the test data is {0:.3f}% of the mean.'.format(test_rse/test_mean*100))
```

The above code resulted in the following output:

```
R squared value of the linear model: 0.017.
Residual standard error of the linear model: 0.039.
The RSE of the linear model is 21.497% of the mean.

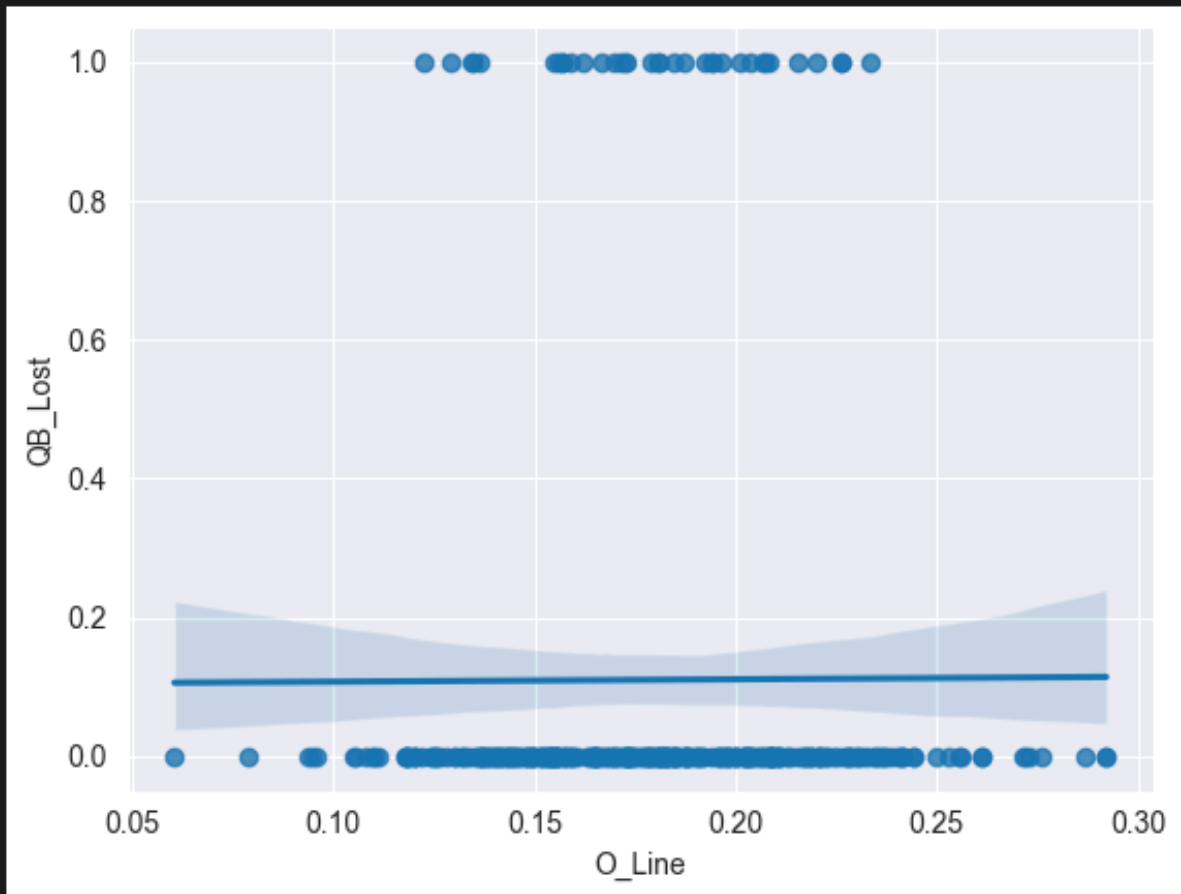
R squared value of the linear model applied to the test data: -0.011.
Residual standard error of the linear model applied to the test data: 0.041.
The RSE of the linear model applied to the test data is 22.765% of the mean.
```

With such small R-squared values and residual standard errors that are such a large percentage of the mean, we can conclude that there is no statistical evidence of a linear correlation between quarterback spending and offensive line spending. This is in line with the scatterplot shown above the mathematical analysis, as there does not appear to be any linear correlation in this plot.

Next, I perform a logistic regression on losing a quarterback ('QB_Lost') as a function of O_Line spending. I first plot a scatterplot of whether or not a team loses their quarterback for

significant time (more than 4 games) as a function of total O-line spending, and plot a logistic regression line using the seaborn function regplot, with the argument `logistic = True`.

```
#plot a scatterplot of QB spending vs O-line spending
plt.figure()
sns.regplot(x = 'O_Line', y = 'QB_Lost', data = X, logistic = True)
plt.show()
✓ 4.3s
```



I then train a logistic regression model on the training data using the statsmodels function `logit`. I predict whether a team will lose their quarterback for significant time on the testing data, and create a confusion matrix to evaluate the performance of the model. I also display the model summary to view its training statistics.

```
#create a model to predict if a quarterback will lose significant time due to injury from o-line spending
oline_v_qblost_model = logit('QB_Lost ~ O_Line', data = train).fit()
print(oline_v_qblost_model.summary())

predicted = oline_v_qblost_model.predict(test).astype('int')
cm = confusion_matrix(test['QB_Lost'], predicted)
cm = pd.DataFrame(cm, columns = ['predicted false', 'predicted true'],
                  index = ['actual false', 'actual true'])
print('\nConfusion Matrix')
display(cm)
```

The confusion matrix shows that the model predicted false for every value. While this is technically fairly accurate at 87.5%, as the model was unable to identify any true positives, both the precision and recall of the model are zero.

Confusion Matrix		
	predicted false	predicted true
actual false	56	0
actual true	8	0

Additionally, we can examine the regression result summary for further statistics. We see that the pseudo R-squared value is very low, and the LLR p-value is very high, both indicating that the model does not perform well.

Logit Regression Results						
=====						
Dep. Variable:	QB_Lost	No. Observations:	254			
Model:	Logit	Df Residuals:	252			
Method:	MLE	Df Model:	1			
Date:	Sun, 27 Oct 2024	Pseudo R-squ.:	0.0003182			
Time:	20:36:39	Log-Likelihood:	-86.004			
converged:	True	LL-Null:	-86.032			
Covariance Type:	nonrobust	LLR p-value:	0.8150			
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	-2.3500	0.969	-2.426	0.015	-4.248	-0.452
O_Line	1.2278	5.241	0.234	0.815	-9.045	11.501
=====						

These findings are in line with the scatterplot shown above, as the logistic regression trend line was unable to create the expected sigmoid shape over the domain of the data. This indicates that losing a quarterback for a significant amount of time due to injury is not statistically significantly correlated with offensive line spending.

Finally, I created a linear model to predict end of season win-loss records using the independent variables discussed previously. I again use the statsmodels ols function to train the model on the training data, predict values with the testing data, and print out statistics on model performance.`

```

#function to create a model string to pass into the ols function for given independent/dependent variables
def create_model_string(ind_var, dep_var):
    string = str(dep_var + ' ~ (')
    for var in ind_var:
        string += var + ' + '
    string = string[:-2] #delete last '+'
    string += ')' + 0'
    return string

#define all independent and dependent variables to be used for the rest of the analysis
ind_vars = ['Strength_of_Schedule', 'C', 'CB', 'DE', 'DT', 'FS', 'G', 'ILB', 'K', 'LB',
            'LS', 'LT', 'OLB', 'P', 'QB', 'RB', 'RT', 'S', 'SS', 'T', 'TE', 'WR', 'OL', 'FB', 'WLB', 'KR']
dep_var = 'Win_Loss'

#create a model using every independent variable
all_var_string = create_model_string(ind_vars, dep_var)
all_var_model = ols(all_var_string, data = train).fit()

rsquared = all_var_model.rsquared
rse = np.sqrt(all_var_model.mse_resid)
train_mean = np.mean(train['Win_Loss'])

#use linear model to predict 0-line spending from qb spending
y_pred = all_var_model.predict(X_test)

```

The performance of the model is shown below.

```

#use residuals to calculate a r-squared value and rse for test group
residuals = y_test - y_pred
test_mean = np.mean(test['Win_Loss'])
dist_mean = test['Win_Loss'] - test_mean
test_rsquared = 1 - np.sum(residuals**2)/np.sum(dist_mean**2)
test_rse = np.std(residuals)

print('R squared value of the linear model: {0:.3f}'.format(rsquared))
print('Residual standard error of the linear model: {0:.3f}'.format(rse))
print('The RSE of the linear model is {0:.3f}% of the mean.'.format(rse/train_mean*100))
print('\nR squared value of the linear model applied to the test data: {0:.3f}'.format(test_rsquared))
print('Residual standard error of the linear model applied to the test data: {0:.3f}'.format(test_rse))
print('The RSE of the linear model applied to the test data is {0:.3f}% of the mean.'.format(test_rse/test_mean*100))

display(all_var_model.summary())

```

✓ 0.1s

```

R squared value of the linear model: 0.144.
Residual standard error of the linear model: 0.188.
The RSE of the linear model is 37.230% of the mean.

R squared value of the linear model applied to the test data: -0.092.
Residual standard error of the linear model applied to the test data: 0.170.
The RSE of the linear model applied to the test data is 35.374% of the mean.

```

Again, we see that the linear model performs poorly, with very low R-squared values and very high residual standard errors on both the training and testing set. This indicates that we do not observe any significant linear correlation between win-loss record and team spending.

As a final piece of analysis, I use the trained linear model to predict the end of season results for the current 2024 season. (Disclaimer: this was proposed as a fun exercise in my project proposal, however as the model shows low performance on both the training and testing data, results taken from this prediction should not be trusted, and any readers of this capstone project should be strongly discouraged from using these results for gambling or any other purpose). I scraped

```
#predict end of season records using linear model
all_data2024 = pd.read_csv('all_data2024.csv')
pred_2024 = all_var_model.predict(all_data2024)
all_data2024['Predicted_Record'] = pred_2024.round(3)
display(all_data2024[['Team', 'Predicted_Record']])
```

the 2024 data using the same methods and code as described in the Data Extraction section of this report, so I will not include the code here. Once the data was collected, I used the model to report a win-loss record for each team. The model predicts that the Chicago Bears will have the best league record, and the Green Bay Packers will have the worst league record. Again, note that the residual standard error found on the test data was 0.170 (same units as the predicted values), which is quite large when compared to the predicted values.

To summarize the analysis process, I created two linear regression models, and one logistic regression model to investigate the research question. These regression tools were appropriate to use because the independent variables were not correlated with each other; in the case of the linear models all variables were continuous, and in the case of the logistic model the dependent variable was a boolean variable. The advantage of using these techniques is that it does not require a lot of data points to detect a strong linear or logistic correlation. One disadvantage of the linear regression technique is that with such a large number of independent variables used for the final model, I was unable to consider interaction terms between variables. Had I considered pairwise interactions between variables, the linear model would have had more degrees of freedom than number of training data points, making it impossible to train the model.

	Team	Predicted_Record
0	Buffalo Bills	0.631
1	Miami Dolphins	0.678
2	New York Jets	0.324
3	New England Patriots	0.593
4	Pittsburgh Steelers	0.424
5	Baltimore Ravens	0.404
6	Cincinnati Bengals	0.599
7	Cleveland Browns	0.531
8	Houston Texans	0.501
9	Indianapolis Colts	0.543
10	Jacksonville Jaguars	0.630
11	Tennessee Titans	0.461
12	Kansas City Chiefs	0.341
13	Denver Broncos	0.453
14	Los Angeles Chargers	0.397
15	Las Vegas Raiders	0.577
16	Washington Commanders	0.592
17	Philadelphia Eagles	0.499
18	Dallas Cowboys	0.372
19	New York Giants	0.560
20	Detroit Lions	0.381
21	Minnesota Vikings	0.435
22	Green Bay Packers	0.280
23	Chicago Bears	0.687
24	Tampa Bay Buccaneers	0.326
25	Atlanta Falcons	0.517
26	New Orleans Saints	0.369
27	Carolina Panthers	0.448
28	Seattle Seahawks	0.641
29	Arizona Cardinals	0.373
30	Los Angeles Rams	0.310
31	San Francisco 49ers	0.442

Data Summary and Implications

Each of the three models were unable to identify a strong linear or logistic regression trend. As such, the result of the analysis is that I am unable to reject the null hypothesis, which states that there is no statistically significant correlation between NFL position salaries and team win-loss records. While the null hypothesis may not be true, a more robust analysis must be required in order to reject it. One major limitation of this analysis was that it did not consider the amount of play time that each player had. For example, while this project considered quarterback injuries and missed time, it did not consider missed time at other positions. If a team spend a large portion of its salary cap on a high valued center who was then injured for a significant part of the season, this would be a confounding variable on the analysis, as the offensive line would likely perform worse without their starting center, while still spending a large portion of the salary cap on these positions.

With regard to the research question, I recommend further analysis as a course of action. The analysis performed for this project was insufficient to reject the null hypothesis, but it is certainly possible that a more in depth analysis could lead to enough evidence to reject it. One direction of further study I would recommend would be to analyze the relationship between positional salaries and in-game statistics. For example, is there a correlation between offensive line salaries and the amount of sacks allowed? Or, is there a correlation between quarterback salaries and passing yards? A second direction of further study I would recommend would be to consider relevant details about each player in conjunction with their salary. For example, the player's age, their injury history, and details about their contract as a whole: whether it is a rookie contract, the length of their contract, and whether they are at the start or end of their current contract.

Submission Files

For my capstone submission, I include:

- This report, titled 'Barnhill_Capstone_Report.pdf'
- The code used for the project, titled 'Barnhill_Capstone_Code.ipynb'
- The final cleaned dataset, titled 'nfl_data.csv'

Sources

¹ A historical table of quarterback contracts can be found at <https://overthecap.com/contract-history/quarterback>.

² Timelines of NFL starting quarterbacks can be found at <https://www.quirkyresearch.com/nfl-starting-quarterback-timelines/>.

³ Screenshot taken from <https://www.pro-football-reference.com/years/2023/>.

⁴ An explanation of the league change to number of games can be found here <https://en.as.com/nfl/why-are-there-17-regular-season-games-in-the-nfl-n-2/>.

⁵ Screenshot taken from https://www.spotrac.com/nfl/miami-dolphins/cap/_/year/2023/sort/cap_total.

⁶ Screenshot taken from https://www.spotrac.com/nfl/miami-dolphins/cap/_/year/2021/sort/cap_total.

⁷ Screenshot taken from <https://www.quirkyresearch.com/nfl-starting-quarterback-timelines/afc-north-starting-quarterback-changes/#Bengals>.

⁸ The github repository used as a starting point for web scraping code can be found here https://github.com/the-data-analyst/python_portfolio_web_scraper-spotrac.